AICTE VERIFIED STUDENT REGISTER

Tech Stack and Repository Specification

Approved MERN-aligned technologies, dependency boundaries, source tree, environment and operating assumptions.

Document: 02-TECH-STACK Version: 1.0 Date: 24 October 2026

Audience: AI coding agent + project owner

Document-control note
This is an implementation specification for a synthetic-data prototype, not an official AICTE standard, production approval, legal opinion, or proof of live government integration.

Source discipline: requirements from the PS 26263 implementation report and Sigma Batch curriculum are distinguished from proposed engineering choices. Any item that depends on a real policy, approval, API, credential trust chain, or deployment environment must be confirmed rather than guessed.

1. Selected implementation stack

Layer	Approved choice	Purpose / constraints
UI	React + JavaScript + Bootstrap	Role-specific screens and responsive layouts. Use plain JavaScript, not TypeScript.
UI build tooling	Vite (proposed)	Development server and production static build. Tool choice is an implementation recommendation, not explicitly listed in the curriculum.
Client routing	React Router (proposed)	Map URLs to protected role-specific pages.
HTTP	Browser fetch API (proposed)	Call REST endpoints; handle loading/error states.
Server	Node.js LTS + Express.js	REST API, business rules, authorization middleware.
Database	MongoDB + Mongoose	Document persistence and schema/model definitions.
Authentication	express-session + connect-mongo (proposed)	Server-side sessions stored in MongoDB; cookie-based browser login.
Password hashing	bcrypt (proposed)	Store only password hashes; never store plaintext passwords.
Security middleware	Helmet + express-rate-limit (proposed)	Baseline headers and request throttling; configuration must be tested.
CSV upload	Multer + a maintained CSV parser (proposed)	Accept bounded demo files and parse records safely.
QR generation	qrcode package (proposed)	Generate demo QR codes; do not treat a URL QR as a signed credential.
Tests	Node.js test runner; Supertest for API tests (proposed)	Repeatable checks of logic, APIs and permissions.
Version control	Git + GitHub	Checkpoints, branches and reviewable changes.
Local runtime	Node processes + MongoDB	Keep first setup understandable; add container orchestration only if needed.

2. Explicitly excluded from the first prototype

- NestJS and TypeScript.
- PostgreSQL, Drizzle ORM and SQL-first schema design.
- Redis, BullMQ and other durable queue infrastructure.
- Keycloak or enterprise SSO integration.
- Python/FastAPI/Isolation Forest model.
- Kubernetes, Helm, production autoscaling and multi-region disaster recovery.
- Live APAAR, ABC, AICTE or DigiLocker APIs and credentials.

Do not add substitutes silently
If a package or service seems necessary, the coding agent must explain the requirement, compare the simplest alternative, identify maintenance/security implications, and wait for approval before introducing a major technology.

3. Suggested repository layout

```
aicte-verified-register/
  client/
    src/
      app/              # routes and shared app shell
      components/       # reusable UI components
      features/         # auth, batches, ledger, conflicts, consent
      pages/            # role-specific screens
      services/         # API client functions
      styles/
  server/
    src/
      config/           # environment and database setup
      models/           # Mongoose models
      routes/           # REST route registration
      controllers/      # HTTP request/response handling
      middleware/       # auth, authorization, error handling
      services/         # business rules and transactions
      validators/       # server-authoritative input checks
      adapters/         # mock APAAR/AICTE/DigiLocker services
      utils/            # logging, hashes, helpers
    tests/
  docs/                 # implementation notes / ADRs
  .env.example
  .gitignore
  README.md
```

The exact folders may be adjusted once, before implementation, if the agent presents a clear reason. Keep routes thin; business rules belong in services and authorization in middleware plus service-level checks.

4. Dependency policy

Dependency	Decision	Required safeguard
mongoose	Use	Schema validation, indexes and explicit connection handling.
express-session + connect-mongo	Use for browser sessions	HttpOnly cookie, appropriate SameSite/Secure, expiry, session rotation after login, server-side invalidation on logout.
bcrypt	Use for password hashing	Asynchronous hash/compare; never log password values.
helmet	Use	Review defaults and test application behaviour.
express-rate-limit	Use	At minimum throttle login and public verify endpoints; avoid locking out legitimate users with simplistic global settings.
multer + CSV parser	Use for small demo uploads	File type and size limits, safe temporary storage, cleanup, malformed-file tests.
qrcode	Use for mock QR	Encode an unguessable demo verification reference, not personal data or secrets.
Any new major dependency	Approval required	Document why built-in Node/Express/Mongoose functionality is insufficient.

5. Environment configuration

```
NODE_ENV=development
PORT=5000
MONGODB_URI=replace_with_local_or_test_uri
SESSION_SECRET=replace_with_long_random_secret
CLIENT_ORIGIN=http://localhost:5173
SESSION_MAX_AGE_MS=3600000
MAX_UPLOAD_MB=5
MAX_ROWS_PER_BATCH=10000
```

The values 5 MB, 10,000 rows and one-hour session lifetime above are proposed developer defaults for a demo, not official policy or proven capacity limits. Tune them only after testing and document any change. Do not commit the real .env file.

6. MongoDB deployment requirement

The certification flow may need a multi-document transaction so the official enrolment, seat reservation, history and audit event remain consistent. MongoDB multi-document transactions require a replica set or sharded cluster. The agent must document and test a supported local replica-set setup or a suitable hosted MongoDB environment before implementing that flow. A standalone local MongoDB server must not be presented as having transaction support for these operations.

7. Operating rules

- The server is authoritative. Client-side validation improves UX but cannot replace server validation.
- Do not process an unbounded CSV or trust its filename/extension alone.
- Avoid adding a durable job queue until actual upload requirements justify it; if using an in-process demo worker, document restart/recovery behaviour.
- No performance target is accepted without an actual measured run and environment details.
- Pin dependency versions through the package lockfile; install current maintained versions at build time and record any security caveat.

8. Source alignment

Sigma Batch curriculum: frontend HTML/CSS/Bootstrap/JavaScript, async/await, AJAX, Git, Node.js, Express, MongoDB, Mongoose, APIs, error handling, authentication and React. The chosen stack keeps to this learning path. The specific packages and security architecture above are implementation recommendations, not claims that every package is directly taught by the curriculum.